

Condition Syntax

Condition is an expression based on operators and operands. Same kind of syntax that you would find in a lot of different applications.

Remarks

In earlier versions of condition syntax, record attributes and functions required a context prefix.

Attributes and record functions were prefixed with `record` , e.g., `record.isDirty()` AND

`record.State = "Open"` . Utility functions were prefixed with `api.` , e.g.,

`api.substring(record.Text, 0, 1)` . This requirement has been removed; attributes and functions in the record context are now accessed directly without prefixes. For example:

`isDirty() AND State = "Open" AND substring(Text, 0, 1) = "A"` .

Operands

Operands can be context data attributes, static values, and functions.

Namespace	Description
record	To specify a data attribute for the current record the record prefix is used. Example: <code>EmpNo = "1001"</code>
parent	Attributes on the parent record is accessed through the parent namespace. Example: <code>parent.CompanyId = "01"</code>
Reference	Attribute on referenced data is accessed through the reference attribute. Example: <code>UrgencyLevelRef.Color = "red"</code>

The current condition field in Page Designer is not a condition editor and have no intellisense capability. Valid record attributes can not be looked up nor can the field validate if a condition attribute reference is valid or not. For this type of information, you need to visit other sources of information, e.g., browse meta in the designer, debug console, application documentation.

Operators

Operator	Description
=	Equals
!=	Is not equal to
>=	Is greater than or equal to
>	Is greater than
<=	Is less than or equal to
<	Is less than
IN	Operator that controls that a value is in the specified list of values Example: <code>EmpNo IN ("1001", "1002")</code>
AND	Logical operator that combines several expressions and require that all sub expressions is successfully evaluated. Example: <code>CompanyId = "01" AND EmpNo = "1001"</code>
OR	Logical operator that requires that one of the sub expressions is successfully evaluated. Example: <code>EmpNo = "1001" OR EmpNo = "1002"</code>
NOT	Logical negation on a <code>Boolean</code> expression, e.g., <code>NOT isValid()</code>

Logical operator groups

When "AND" and "OR" operators are combined in a condition parenthesis are required to enforce a specific evaluation order.

Example: `CompanyId = "01" AND (EmpNo = "1001" OR EmpNo = "1002")`

Values

In configurations, value types are not localized and follow a fixed syntax regardless of where the application is run.

Type	Description
string	A string value is enclosed within double quotes, e.g., "this is a string"
number	Number values are written using digits, with the decimal point represented by a dot, e.g., 3, -4, 0.5
date	Date values follow the ISO-8601 format, e.g., 2401-04-01 (YYYY-MM-DD)
date time	Date-time values also follow the ISO-8601 standard, with a T as the delimiter between the date and time, e.g., 2401-04-01T23:01:43 (YYYY-MM-DDThh:mm:ss)
regular expression	Regular expression pattern syntax is not validated in the editor. Instead, the executing browser enforces any syntax requirements during the matching process. Different browsers offer varying levels of support for common regex syntax, meaning that a pattern which works in one browser might not behave the same way in another. This can result in syntax errors or subtle differences, even on the same operating system. To ensure consistent behavior, it's important to thoroughly test your Regular Expressions across all target browsers and versions. Flags: i (Ignore case) The i flag makes the regular expression case-insensitive, allowing it to match both uppercase and lowercase letters without distinction. m (Multiline mode) The m flag changes the behavior of the anchors ^ and \$. By default, ^ matches the start of the string, and \$ matches the end. With the m flag, ^ matches the start of each line, and \$ matches the end of each line within a multi-line string. s (Dot all mode) The s flag allows the dot (.) in the regular expression to match newline characters (\n). By default, the dot does not match newlines, meaning it only matches any character except newlines. With s, the dot matches every character, including newlines. Example: <code>/test/i</code> - Matches "test" anywhere in a string regardless of case.

Functions

Function	Description
<code>contains(value: string, list: string[])</code>	Check if a value exists in a list of values. Example: <code>contains("Barcode", Objgrants)</code>
<code>count([attribute: string, attribute: string ,...])</code> <code>count([attribute: string, attribute: string ,...], condition: string)</code>	Count the number of records in the current entity set. Examples: Count all records <code>count(["*"]) > 0</code> Count records with distinct value for an attribute. <code>count(["State"]) > 0</code> Count records where the combination of two values is distinct <code>count(["State", "Resolution"]) > 0</code> Count records that meet a condition. For nested strings double quotes must be escaped with backslash <code>count(["State"], "State IN (\\"new\\", \\"open\\")") > 0</code>
<code>substring(value: string, start: number, length: number)</code>	Extract a sub string from a string. Start at given start position and extracts the given number of characters. Example: <code>substring(Flags, 5, 1) = "X"</code>
<code>sum</code>	Returns a sum of all the values in an entity set for a given attribute. Example: <code>sum("AvailableToReserve") > 0</code>
<code>match(value: string, pattern: regex)</code>	Tests if a string matches a Regular Expression. Example: <code>match("Barcode", /source.*code/i)</code> For regex syntax details, see regular expressions under Values.
<code>dateDiff(unit: dateUnit, start: Date, end: Date)</code>	Returns the difference between two dates based on specified date unit. The resulting difference is always a positive integer Date unit: year, month, day, hour, minute, second. Date formats: YYYY-MM-DDThh:mm:ss, YYYY-MM-DD, \<attribute>, Date Context Substitution Variable

Function	Description
	Example: <code>dateDiff(year, 2000-01-01, CreationDate) > 5</code> <code>dateDiff(day, CreationDate, 2024-01-01T12:30:00) < 200</code>

Record functions

Record functions are a set of functions that will operate directly on the current record.

Function	Description
isNew()	Returns true if the record is new, i.e., is not previously save to the database
isDirty()	Returns true if the record contains any changed unsaved values.
isValid()	Returns true if all values on the record contains valid values.

Context Substitution Variables (CSV)

Overview

This section provides a comprehensive list of context substitution variables that can be used to dynamically reference specific values, e.g., dates and system parameters. These variables automatically substitute with the corresponding value when used, allowing for flexible and context-sensitive operations.

Date Variables

Variable	Description	Example (if today is 2024-08-15)
#TODAY#	Substitutes with the current date (today's date).	2024-08-15
#TOMORROW#	Substitutes with the date of the next day.	2024-08-16
#YESTERDAY#	Substitutes with the date of the previous day.	2024-08-14

Variable	Description	Example (if today is 2024-08-15)
#START_OF_THIS_WEEK#	Date of the first day of the current week.	2024-08-12 (if week starts on Monday)
#END_OF_THIS_WEEK#	Date of the last day of the current week.	2024-08-18 (if week ends on Sunday)
#START_OF_THIS_MONTH#	Date of the first day of the current month.	2024-08-01
#END_OF_THIS_MONTH#	Date of the last day of the current month.	2024-08-31
#START_OF_THIS_YEAR#	Date of the first day of the current year.	2024-01-01
#END_OF_THIS_YEAR#	Date of the last day of the current year.	2024-12-31
#START_OF_LAST_WEEK#	Date of the first day of the previous week.	2024-08-05 (if week starts on Monday)
#END_OF_LAST_WEEK#	Date of the last day of the previous week.	2024-08-11 (if week ends on Sunday)
#START_OF_LAST_MONTH#	Date of the first day of the previous month.	2024-07-01
#END_OF_LAST_MONTH#	Date of the last day of the previous month.	2024-07-31
#START_OF_LAST_YEAR#	Date of the first day of the previous year.	2023-01-01
#END_OF_LAST_YEAR#	Date of the last day of the previous year.	2023-12-31
#START_OF_NEXT_WEEK#	Date of the first day of the next week.	2024-08-19 (if week starts on Monday)

Variable	Description	Example (if today is 2024-08-15)
#END_OF_NEXT_WEEK#	Date of the last day of the next week.	2024-08-25 (if week ends on Sunday)
#START_OF_NEXT_MONTH#	Date of the first day of the next month.	2024-09-01
#END_OF_NEXT_MONTH#	Date of the last day of the next month.	2024-09-30
#START_OF_NEXT_YEAR#	Date of the first day of the next year.	2025-01-01
#END_OF_NEXT_YEAR#	Date of the last day of the next year.	2025-12-31
#FIRST_DAY_OF_THIS_WEEK#	Synonym for #START_OF_THIS_WEEK# .	2024-08-12
#LAST_DAY_OF_THIS_WEEK#	Synonym for #END_OF_THIS_WEEK# .	2024-08-18
#FIRST_DAY_OF_THIS_MONTH#	Synonym for #START_OF_THIS_MONTH# .	2024-08-01
#LAST_DAY_OF_THIS_MONTH#	Synonym for #END_OF_THIS_MONTH# .	2024-08-31
#FIRST_DAY_OF_THIS_YEAR#	Synonym for #START_OF_THIS_YEAR# .	2024-01-01
#LAST_DAY_OF_THIS_YEAR#	Synonym for #END_OF_THIS_YEAR# .	2024-12-31
#FIRST_DAY_OF_LAST_WEEK#	Synonym for #START_OF_LAST_WEEK# .	2024-08-05
#LAST_DAY_OF_LAST_WEEK#	Synonym for #END_OF_LAST_WEEK# .	2024-08-11

Variable	Description	Example (if today is 2024-08-15)
#FIRST_DAY_OF_LAST_MONTH#	Synonym for #START_OF_LAST_MONTH# .	2024-07-01
#LAST_DAY_OF_LAST_MONTH#	Synonym for #END_OF_LAST_MONTH# .	2024-07-31
#FIRST_DAY_OF_LAST_YEAR#	Synonym for #START_OF_LAST_YEAR# .	2023-01-01
#LAST_DAY_OF_LAST_YEAR#	Synonym for #END_OF_LAST_YEAR# .	2023-12-31
#FIRST_DAY_OF_NEXT_WEEK#	Synonym for #START_OF_NEXT_WEEK# .	2024-08-19
#LAST_DAY_OF_NEXT_WEEK#	Synonym for #END_OF_NEXT_WEEK# .	2024-08-25
#FIRST_DAY_OF_NEXT_MONTH#	Synonym for #START_OF_NEXT_MONTH# .	2024-09-01
#LAST_DAY_OF_NEXT_MONTH#	Synonym for #END_OF_NEXT_MONTH# .	2024-09-30
#FIRST_DAY_OF_NEXT_YEAR#	Synonym for #START_OF_NEXT_YEAR# .	2025-01-01
#LAST_DAY_OF_NEXT_YEAR#	Synonym for #END_OF_NEXT_YEAR# .	2025-12-31

Usage Notes

- **Flexibility:** These variables can be used in most places as a substitute for an attribute, where the value needs to be dynamically calculated and substituted in runtime. All date variables also support offsets.

Offset syntax

```
#<identifier>#<offset>
```


Explanation:

- #\<identifier># - The input must start and end with a #, enclosing a valid CSV identifier.
- \<offset> - (Optional) A signed integer adjustment:
 - Starts with a + or -.
 - Followed by a non-zero number (e.g., +10, -25).
 - Cannot be 0.

Example: Adding one day to the current day

```
#TODAY#+1
```

Syntax change:

For releases 24.2.0 through 24.2.2, the syntax for offsets was denoted as #<identifier><offset>#. However, starting with version 24.2.3, the syntax for user input was standardized to align with the format utilized in Search Fields.

Note : *Configurations saved with the deprecated syntax remain supported at runtime. However, any modifications to these configurations require an update to the syntax, as the input fields no longer accepts the old format.*

- **Case Sensitivity:** Variable names are typically case-sensitive
- **Date Format:** Date CSV values will be serialized as ISO date string. '2024-01-01'. This means that one can not directly compare an attribute of timestamp '2024-01-01T03:01:00'. Where this is needed for example to compare a creation date with #TODAY# use
toDate(Created) = #TODAY#